



Facultad de Ingeniería

**Fundamentos de Ingeniería de Software
020 - 86**

Ana Karina Roa Mora-20232020118 Juan

Pablo Galindo Florez - 20231020230

**Stevens Camilo Llanos Acero –
20231020221**

Mayo 21, 2026

QueBoleta

Descripción General

QueBoleta es una aplicación web desarrollada para la gestión y venta de entradas de eventos. El sistema permite a los usuarios registrarse, iniciar sesión, consultar eventos disponibles y comprar entradas digitales con códigos QR únicos para el acceso a los eventos. Además, cuenta con roles específicos como administrador, encargado de gestionar eventos y reportes, y personal de acceso, responsable de validar las entradas mediante QR. La aplicación fue desarrollada utilizando Spring Boot, MySQL y autenticación JWT, siguiendo una arquitectura basada en API REST para garantizar seguridad, organización y escalabilidad del sistema.

Requerimientos

- **Elicitación de requerimientos:** Información obtenida gracias a entrevistas realizadas al cliente, separa por segmentos de interés.

OBJETIVOS DEL NEGOCIO:

1. **¿Cómo se gestionan actualmente los eventos y qué dificultades presenta el proceso actual?**

Actualmente se gestionan los eventos de forma manual: se usan hojas de Excel para controlar asistentes y ventas, y la promoción se hace por redes sociales y WhatsApp.

Las principales dificultades son:

- Errores en el conteo de entradas vendidas.
- Falta de control claro del aforo.
- Dificultad para consolidar reportes de ventas.
- Mucho tiempo invertido en tareas administrativas.
- Riesgo de pérdida de información.

2. **¿Qué tipo de eventos manejarán?**

La plataforma maneja conciertos, obras de teatro y conferencias académicas. No se contemplan eventos deportivos en esta versión.

3. ¿Planeas operar en una sola ciudad o en varias?

Solo planeamos operar en Bogotá por el momento.

4. ¿Cómo se evaluará que el sistema cumple con lo esperado?

Se considerará que el sistema es exitoso si:

- Se pueden crear eventos sin errores.
- Las entradas se venden correctamente.
- El pago se procesa sin fallos.
- El control de acceso funciona sin duplicidades.
- Los reportes coinciden con las ventas reales.
- El sistema es fácil de usar para el equipo.

USUARIOS Y ROLES:

5. ¿Quién administrará el sistema?

El sistema será administrado por:

- Un organizador de eventos.
- Una persona de apoyo encargada de logística o ventas.

6. ¿Qué tipos de usuarios utilizarán la aplicación y qué podrá hacer cada uno?

A. Administrador

- Crear, editar y eliminar eventos.
- Ver reportes de ventas.
- Configurar precios y aforo.

B. Personal de acceso

- Validar entradas mediante código QR o código numérico.
- Ver lista de asistentes.

C. Cliente (asistente)

- Ver eventos disponibles.
- Comprar entradas.
- Recibir comprobante digital.

7. ¿Qué mecanismos de autenticación y autorización se requieren?

Se solicita:

- Inicio de sesión con correo y contraseña.
- Recuperación de contraseña por correo.
- Control de acceso por roles (administrador y personal de acceso).

GESTION DE EVENTOS:

8. ¿Qué información desea mostrar (lugar del evento, aforo, sinopsis del artista, precio, stock, etc.)?

Se debe mostrar

- Nombre del artista.
- Nombre del evento.
- Lugar del evento.
- Fecha y hora del evento.

- Información del evento (sinopsis)
- Precios y localidades.

9. ¿La aplicación debe incluir herramientas de promoción?

Así es, se desea:

- Generación de enlace compartible del evento.
- Código de descuento.
- Imagen y descripción del evento.

PAGOS Y FACTURACION:

10. ¿La aplicación debe permitir la venta de entradas? Si es así, ¿qué métodos de pago deben integrarse? ¿Desea manejar cancelaciones, devoluciones o cambios de fecha?

Sí, debe permitir venta online.

Métodos de pago:

- Tarjeta débito/crédito.

Cancelaciones y devoluciones:

- No habrá reembolso automático.
- En caso de cancelación del evento, el reembolso será gestionado manualmente. •

En caso de cambio de fecha, la entrada seguirá siendo válida.

11. ¿La plataforma calculara impuestos automáticamente?

Si, el sistema añadirá automáticamente un porcentaje de impuesto previamente configurado al valor de entrada.

12. ¿Desea que genere factura electrónica?

Solo se generará un comprobante por correo electrónico.

ENTRADAS Y ACCESO:

13. ¿Las entradas serán digitales, físicas o ambas?

Solo serán digitales.

14. ¿Se usarán código QR o código de barras?

Se usará código QR generado automáticamente por el sistema.

15. ¿Se permite la transferencia de entradas entre personas?

No, por el momento la entrada quedará asociada al comprador y no se permitirá la transferencia.

COMUNICACION CON CLIENTES:

16. ¿Desea que la aplicación envíe notificaciones o recordatorios a los asistentes?

Sí, pero de forma básica:

- Confirmación de compra por correo.
- Recordatorio automático 24 horas antes del evento.

17. ¿Se enviarán por correo o SMS?

Si se enviara un correo electrónico automático con el resumen de la compra y el código

QR. REPORTES Y ANALITICA:

18. ¿Qué tipo de reportes o estadísticas necesita el sistema?

El sistema necesita:

- Total de entradas vendidas por evento.
- Ingresos generados por evento.
- Número de asistentes confirmados.
- Cantidad de entradas disponibles.

19. ¿Los administradores podrán ver las estadísticas en tiempo real?

Si, podrán consultar información básica en tiempo real como número de entradas vendidas, disponibilidad y el total de ingresos por evento, mediante un panel sencillo dentro de la plataforma.

20. ¿Cuántos eventos y usuarios simultáneos estima manejar en momentos de alta demanda?

Se espera manejar:

- Entre 5 y 15 eventos activos al mismo tiempo.
- Máximo estimado: 300–500 usuarios simultáneos durante la venta inicial de entradas.

EXPERIENCIA DE USUARIO

21. ¿Cómo desea que sea el proceso de compra para el cliente?

Se desea que el proceso sea simple e intuitivo, con pocos pasos, permitiendo seleccionar entradas, visualizar el resumen de pago y finalizar la compra de manera rápida y clara.

22. ¿El sistema debe almacenar y permitir la consulta del historial de transacciones realizadas por cada usuario?

Sí, cada usuario tendrá acceso a su historial de transacciones y entradas compradas dentro de la plataforma, en el cual podrá verificar los datos, fechas de cada evento que el usuario haya comprado.

DISEÑO Y PRESENTACIÓN

23. ¿Qué estilo visual desea que tenga la plataforma?

Se desea un diseño moderno y atractivo, con una estética asociada a conciertos y eventos culturales, utilizando colores oscuros combinados con botones cromáticos para acciones importantes, principalmente de colores básicos, en el cual se vea ese contraste más naturalmente y mayor acceso visualmente.

24. ¿La plataforma debe ser adaptable a dispositivos móviles?

Sí, la plataforma debe ser completamente responsive para garantizar una buena experiencia desde celulares, tablets y computadores.

CONTROL Y SEGURIDAD

25. ¿Se debe implementar algún mecanismo para evitar compras masivas o reventa?

Sí, se establecerá un límite máximo de entradas por usuario para evitar la compra excesiva y la posible reventa no autorizada.

26. ¿El sistema debe registrar actividad importante dentro de la plataforma?

Sí, el sistema almacenará registros básicos de actividad como compras realizadas,

validaciones de entrada y modificaciones realizadas por el administrador.

FUNCIONALIDADES FUTURAS

27. ¿Se contempla ampliar la plataforma a otras ciudades en el futuro?

Sí, aunque inicialmente operará solo en Bogotá, el sistema debe estar diseñado para permitir expansión a otras ciudades más adelante.

28. ¿El sistema debe estar preparado para soportar un aumento en la cantidad de eventos y usuarios en el futuro?

Sí, la plataforma deberá desarrollarse con una estructura escalable que permita aumentar la cantidad de eventos y usuarios sin afectar su rendimiento.

SOPORTE Y MANTENIMIENTO

29. ¿Se requiere algún tipo de soporte técnico para los usuarios?

Sí, se incluirá una sección de contacto o soporte donde los usuarios puedan enviar consultas relacionadas con compras o eventos.

30. ¿El sistema debe permitir realizar copias de seguridad de la información?

Sí, se debe garantizar respaldo periódico de la información para evitar pérdida de datos. En el dado caso que ocurra algo con la aplicación o cuenta, el usuario tenga su respaldo de las compras que haya hecho.

- **Formalización de requerimientos:** Requerimientos obtenidos luego del análisis de la información.

REQUISITOS FUNCIONALES

1. Gestión de Usuarios y Roles

RF1.1 El sistema deberá permitir el registro e inicio de sesión mediante correo electrónico y contraseña.

RF1.2 El sistema deberá permitir la recuperación de contraseña vía correo electrónico.

RF1.3 El sistema deberá gestionar roles de usuario:

- Administrador
- Personal de acceso
- Cliente

RF1.4 El sistema deberá restringir funcionalidades según el rol asignado.

2. Gestión de Eventos

RF2.1 El administrador podrá crear, editar y eliminar eventos. **RF2.2** El sistema deberá permitir configurar:

- Nombre del evento
- Nombre del artista
- Lugar
- Fecha y hora
- Sinopsis
- Precios por localidad
- Aforo máximo
- Imagen del evento

RF2.3 El sistema deberá generar un enlace compartible para cada evento. **RF2.4** El sistema deberá permitir la creación y aplicación de códigos de descuento. **RF2.5** El sistema deberá controlar automáticamente el stock de entradas según el aforo configurado.

3. Venta de Entradas

RF3.1 El cliente podrá visualizar eventos disponibles. **RF3.2** El cliente podrá seleccionar cantidad de entradas (respetando límite máximo por usuario).

RF3.3 El sistema deberá calcular automáticamente:

- Subtotal
- Impuestos configurados

- Total a pagar

RF3.4 El sistema deberá permitir pago mediante:

- Tarjeta débito
- Tarjeta crédito

RF3.5 El sistema deberá generar una entrada digital con código QR único. **RF3.6** El sistema enviará automáticamente un correo de confirmación con:

- Resumen de compra
- Código QR
- Datos del evento

RF3.7 En caso de cambio de fecha, la entrada seguirá siendo válida. **RF3.8** En caso de cancelación del evento, el administrador gestionará manualmente los reembolsos.

4. Control de Acceso

RF4.1 El personal de acceso podrá validar entradas mediante código QR. **RF4.2** El sistema deberá impedir la validación duplicada de una misma entrada. **RF4.3** El sistema deberá mostrar la lista de asistentes confirmados. **RF4.4** El sistema registrará la hora de validación de cada entrada.

5. Reportes y Analítica

RF5.1 El sistema deberá mostrar en tiempo real:

- Total de entradas vendidas por evento
- Ingresos generados
- Entradas disponibles
- Número de asistentes confirmados

RF5.2 El administrador podrá consultar reportes consolidados por evento.

6. Historial y Gestión del Cliente

RF6.1 El cliente podrá acceder a su historial de compras. **RF6.2** El cliente podrá visualizar sus entradas activas y pasadas.

7. Notificaciones y Comunicación

RF7.1 El sistema enviará confirmación automática por correo electrónico tras la compra.

RF7.2 El sistema enviará recordatorio automático 24 horas antes del evento. **RF7.3** El sistema incluirá un formulario de contacto o soporte para consultas.

8. Seguridad y Control

RF8.1 El sistema deberá establecer un límite máximo de entradas por usuario. **RF8.2** El sistema deberá registrar actividades importantes como:

- Compras
- Validaciones
- Modificaciones de eventos

9. Respaldo y Mantenimiento

RF9.1 El sistema deberá realizar copias de seguridad periódicas de la información. **RF9.2** El sistema deberá permitir la recuperación de datos en caso de fallos.

REQUISITOS NO FUNCIONALES

1. Rendimiento

RNF1.1 El sistema deberá soportar entre 300 y 500 usuarios simultáneos durante alta demanda.

RNF1.2 El tiempo de respuesta en procesos de compra no deberá superar los 3 segundos en condiciones normales. **RNF1.3** Los reportes en tiempo real deberán actualizarse sin recargar completamente la página.

2. Escalabilidad

RNF2.1 El sistema deberá estar diseñado con arquitectura escalable. **RNF2.2** Deberá permitir expansión futura a otras ciudades sin rediseño estructural. **RNF2.3** Deberá permitir aumentar la cantidad de eventos activos sin afectar el rendimiento.

3. Seguridad

RNF3.1 Las contraseñas deberán almacenarse cifradas. **RNF3.2** La comunicación deberá realizarse bajo protocolo HTTPS. **RNF3.3** El sistema deberá proteger contra validaciones duplicadas de QR. **RNF3.4** El sistema deberá registrar logs de actividad.

4. Usabilidad

RNF4.1 El proceso de compra deberá realizarse en pocos pasos, de forma intuitiva.

RNF4.2 La interfaz deberá ser clara y fácil de usar para el equipo administrativo. **RNF4.3** El sistema deberá ser completamente responsive (móvil, tablet y escritorio).

5. Diseño

RNF5.1 La plataforma deberá tener un diseño moderno orientado a eventos culturales.

RNF5.2 Se utilizará una paleta de colores oscuros con botones cromáticos para acciones principales.

6. Disponibilidad

RNF6.1 El sistema deberá garantizar disponibilidad durante periodos de venta activa.

RNF6.2 Se deberán realizar respaldos periódicos para evitar pérdida de información.

7. Mantenibilidad

RNF7.1 El sistema deberá desarrollarse con código modular y documentado. **RNF7.2** Deberá permitir futuras mejoras sin afectar funcionalidades actuales.

Metodología API

El sistema QueBoleta implementa una arquitectura basada en una API RESTful desarrollada con Spring Boot. Esta metodología permite la comunicación entre el frontend desarrollado en React y el backend mediante peticiones HTTP utilizando intercambio de datos en formato JSON. Se emplea autenticación JWT para mantener un modelo stateless, mejorando la seguridad y escalabilidad del sistema.

Diagramas – Implementación

➤ Casos de uso

• *Gestión de Eventos*

El caso de uso relacionado con los eventos fue implementado principalmente en las clases EventoController, EventoService y EventoRepository. El organizador o administrador puede crear eventos proporcionando información como nombre, artista, lugar, fecha, precio y cantidad de entradas disponibles. El sistema almacena estos datos en la entidad Evento, genera automáticamente un enlace único para el evento y controla la disponibilidad de entradas mediante el atributo disponibilidad. También se implementaron funcionalidades para editar eventos, buscar eventos por nombre, consultar eventos activos y cancelar eventos cambiando su estado a CANCELADO. Durante una compra, el sistema actualiza automáticamente el stock de entradas disponibles y genera los códigos QR correspondientes para cada entrada adquirida.

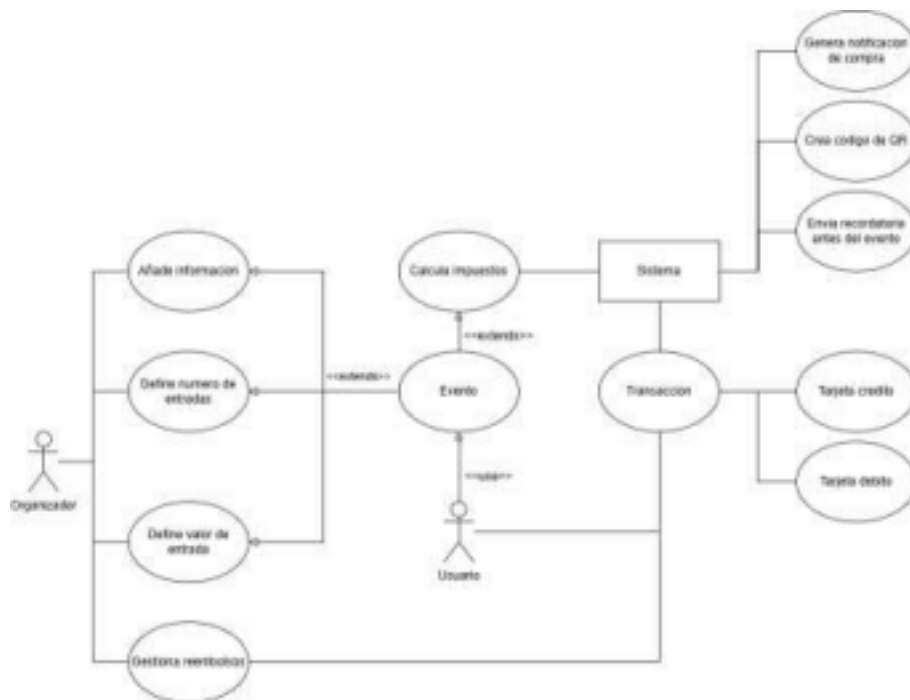


Diagrama de Casos de Uso – Gestión de Eventos

• *Iniciación de sesión*

El caso de uso de inicio de sesión fue implementado utilizando Spring Security y autenticación JWT. El usuario puede registrarse mediante el endpoint `/api/auth/registro`, proporcionando nombre, correo, contraseña y rol. Posteriormente puede iniciar sesión usando `/api/auth/login`. El sistema valida las credenciales mediante AuthenticationManager y CustomUserDetailsService, que consulta los usuarios almacenados en la base de datos.

Una vez autenticado, se genera un token JWT usando la clase JwtUtils, el cual contiene la información del usuario y su rol. Este token debe enviarse en cada petición protegida mediante el encabezado Authorization: Bearer TOKEN. El filtro JwtAuthFilter intercepta las solicitudes, valida el token y autoriza el acceso según el rol del usuario.

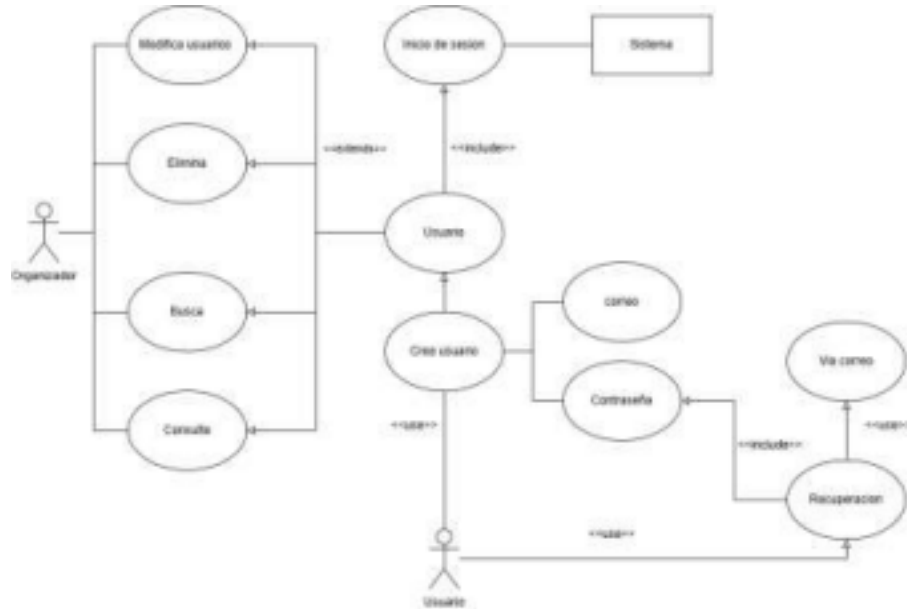


Diagrama de Casos de Uso – Inicio de Sesión

• Reportes

El caso de uso de reportes fue implementado mediante la entidad Reporte y las relaciones existentes con los eventos y transacciones. El sistema puede almacenar información relacionada con entradas vendidas e ingresos generados por cada evento. Aunque actualmente la generación de reportes se encuentra parcialmente implementada, la estructura ya permite registrar estadísticas de ventas e ingresos totales. La clase Administrador contiene métodos como verReportes() y agregarReporte(), lo que permite asociar reportes a los administradores. Además, las transacciones almacenadas en la tabla transaccion sirven como base para calcular datos como cantidad de boletos vendidos, ingresos totales y estado de las compras, lo que facilita una futura expansión del módulo de reportes y análisis del sistema.

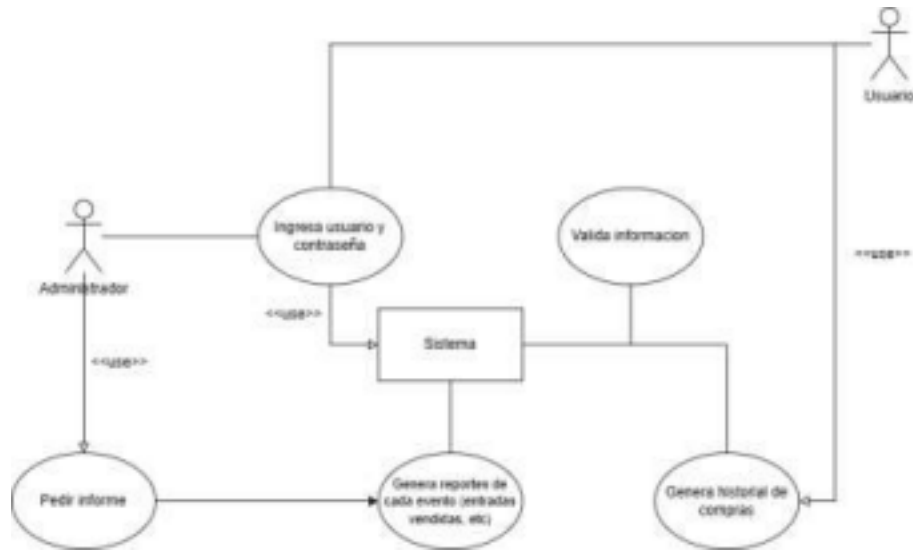


Diagrama de Casos de Uso – Reportes

➤ Diagrama de contexto

El diagrama de contexto se implementó mediante una arquitectura cliente-servidor basada en API REST. El “Sistema de gestión de eventos” corresponde a la aplicación Spring Boot desarrollada en el backend. Los actores externos del diagrama, como el usuario, el administrador, el organizador y el sistema de pagos, interactúan con el sistema mediante endpoints REST definidos en los controladores (AuthController, EventoController, CompraController, entre otros). El usuario puede consultar eventos y realizar compras; el administrador puede crear, editar o cancelar eventos; y el sistema de pagos fue representado mediante el servicio PagoService, encargado de manejar la lógica de procesamiento de pagos. Toda la comunicación se realiza mediante solicitudes HTTP y respuestas JSON, utilizando autenticación JWT para proteger las operaciones privadas del sistema.



Diagrama de Contexto

➤ Diagrama de clases

El diagrama de clases fue implementado en el proyecto utilizando entidades Java con JPA y Spring Boot. La clase abstracta `Usuario` funciona como clase base y utiliza herencia `SINGLE_TABLE`, permitiendo que `Cliente`, `Administrador` y `PersonalAcceso` compartan la misma tabla en la base de datos, diferenciándose mediante el atributo `rol`. Las relaciones mostradas en el diagrama también fueron implementadas mediante anotaciones JPA como `@OneToMany` y `@ManyToOne`; por ejemplo, un `Cliente` puede tener múltiples `Entrada`, mientras que una `Transaccion` se relaciona con un `Usuario` y un `Evento`. Los métodos planteados en el modelo UML, como `generarQR()`, `consultarDisponibilidad()`, `realizarCompra()` o `solicitarReembolso()`, fueron desarrollados dentro de las entidades y servicios correspondientes. Además, los enumeradores del diagrama (`EstadoEvento`, `EstadoEntrada`, `MetodoPago` y `TipoNotif`) fueron implementados como enums de Java para controlar estados y tipos dentro de la aplicación.

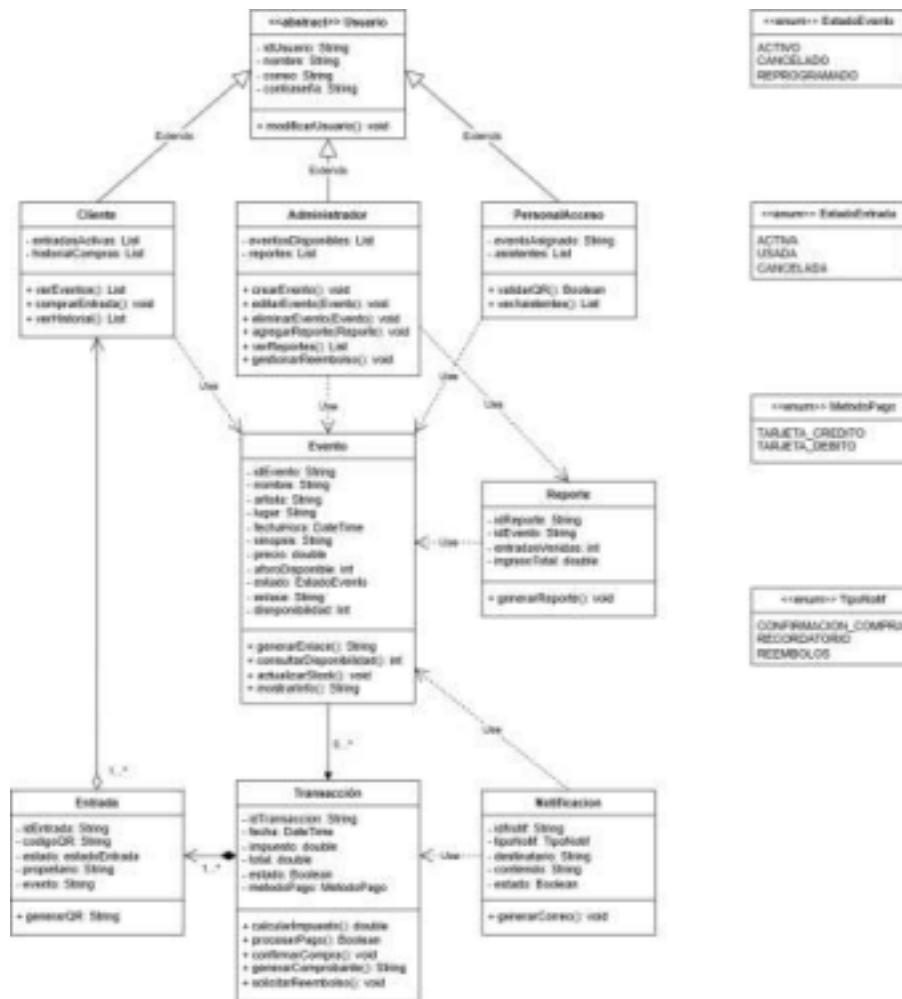


Diagrama de Clases

➤ Diagrama de secuencia

El diagrama de secuencia muestra el flujo completo de compra a través de cinco participantes: Cliente, Página Web, Sistema de Ventas, Punto de Pago y Banco. En el proyecto, ese flujo quedó implementado de la siguiente manera: el Cliente envía su solicitud a través del endpoint POST /api/compras del CompraController, que representa la interacción con la Página Web; este delega en CompraService.realizarCompra(), que actúa como el Sistema de Ventas, verificando disponibilidad del evento y creando la Transaccion. Desde allí se simula la comunicación con el Punto de Pago y el Banco a través de PagoService, que valida los datos de tarjeta del CompraRequest y retorna un resultado de aprobación, tras lo cual CompraService confirma la compra, descuenta el stock del evento, genera las Entradas con su código QR mediante entrada.generarQR(), y dispara el NotificacionService para enviar la boleta digital al correo del cliente, devolviendo finalmente un CompraResponse con los códigos QR, el resumen financiero y el mensaje de confirmación.

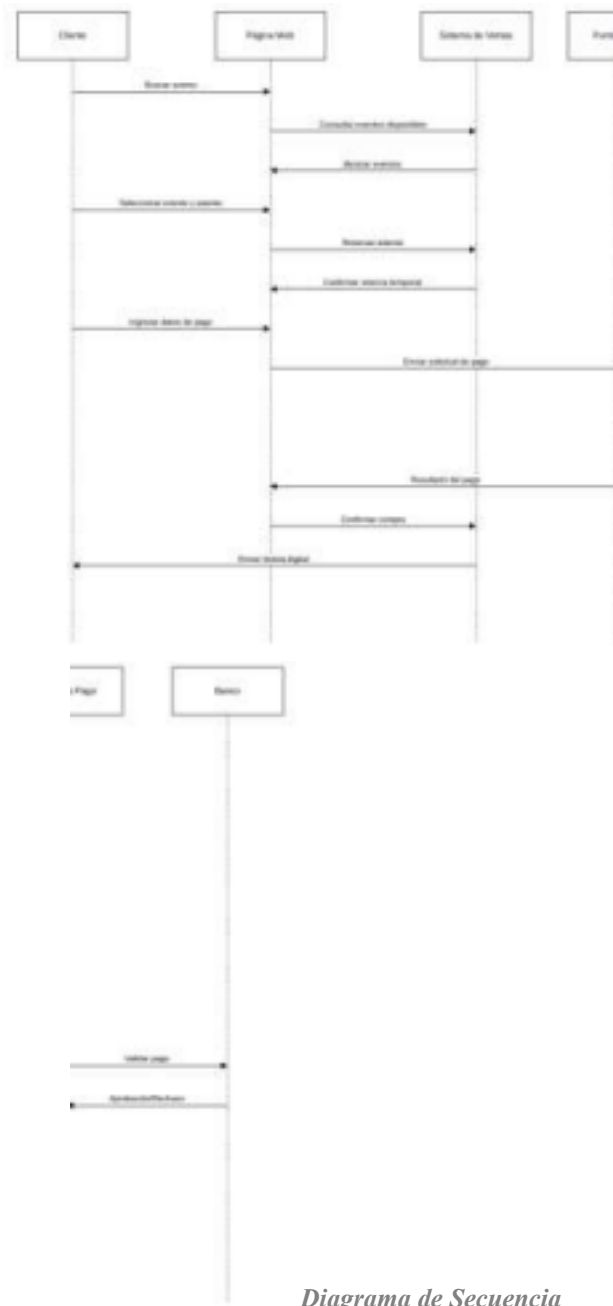


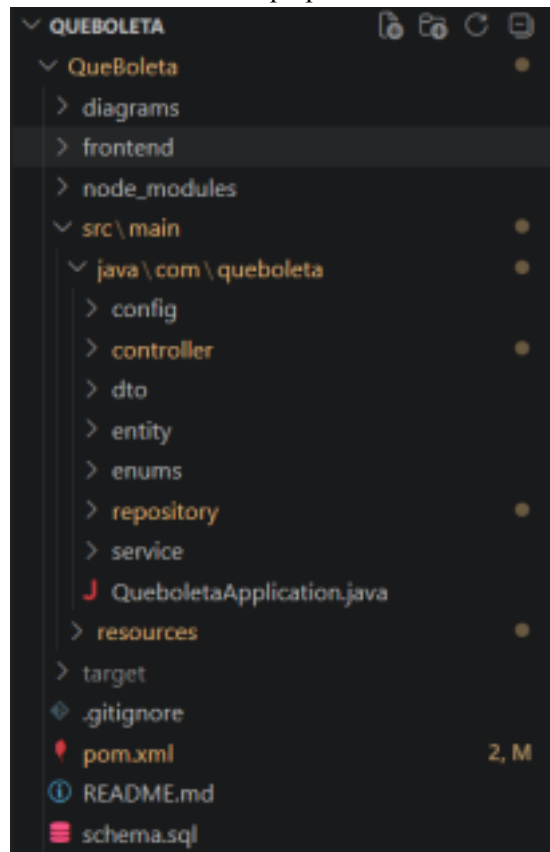
Diagrama de Secuencia

Arquitectura del proyecto

El proyecto sigue una arquitectura por capas:

Controller → Service → Repository → Database

Y las clases se encuentran contenidas en paquetes:



Empaquetado de clases – Visual Studio Code

Tecnologías utilizadas

Para el desarrollo del proyecto se necesitó de:

- Java 17+ : Lenguaje principal
- Spring Boot: Framework Backend.
- Spring Security: Seguridad y autenticación.
- JWT: Manejo de tokens.
- Spring Data JPA: Persistencia.
- Hibernate: ORM.
- MySQL: Base de datos.
- Maven: Gestión de dependencias.

- Swagger/OpenAPI: Documentación API:

Pruebas

1. Prueba funcional — Login

Objetivo

Verificar que un usuario pueda iniciar sesión correctamente.

Pasos

1. Ir a /login
2. Ingresar correo válido
3. Ingresar contraseña válida
4. Presionar "Iniciar sesión"

Resultado esperado

El sistema redirige al Home y guarda el token JWT.

Resultado obtenido

El usuario ingresó correctamente y el token se almacenó en localStorage.

2. Prueba funcional — Compra de entradas

Objetivo

Verificar que un usuario pueda comprar entradas.

Pasos

1. Seleccionar un evento
2. Agregar al carrito
3. Finalizar compra

Resultado esperado

La entrada se guarda en la base de datos y aparece en el perfil.

Resultado obtenido

La entrada fue creada correctamente y se generó un código QR único.

3. Prueba funcional — Panel administrador

Objetivo

Validar que un administrador pueda crear, editar y eliminar eventos.

Pasos

1. Iniciar sesión como ADMINISTRADOR
2. Crear un evento
3. Editar el evento
4. Eliminar el evento

Resultado esperado

El evento se refleja correctamente en el sistema.

Resultado obtenido

Las operaciones CRUD funcionaron correctamente.

4. Prueba unitaria — Generación QR

Clase probada

`Entrada.java`

Método probado

`generarQR()`

Objetivo

Verificar que cada entrada genere un código QR único.

Resultado esperado

El QR contiene:

- prefijo QB
- ID entrada
- timestamp

Resultado obtenido

El método generó códigos únicos correctamente.

5. Prueba de estrés/simple carga

Objetivo

Evaluar el comportamiento del sistema ante múltiples consultas.

Procedimiento

1. Abrir múltiples pestañas
2. Consultar eventos repetidamente
3. Realizar varias compras consecutivas

Resultado esperado

El sistema mantiene estabilidad y responde correctamente.

Resultado obtenido

El backend respondió correctamente sin caídas ni pérdida de información.

Tipo	Módulo	Resultado
Funcional	Login	Exitoso
Funcional	Compra entradas	Exitoso

Funcional	CRUD eventos	Exitoso
Unitaria	Generar QR	Exitoso
Estrés	Múltiples consultas	Exitoso

Acceso al proyecto

Enlace al repositorio: <https://github.com/Juangf1222/QueBoleta.git>